Estructuras de programación: guía para el Parcial 2

Clase 13 · Jueves 22 de octubre de 2026 · Aula

Objetivo de la sesión

Leer un enunciado, reconocer qué estructura pide y escribir su esqueleto sin dudar.

Decisiones

`if`, `if` / `else`, `elif`, `if` anidados, `and` / `or` / `not`.

Ciclos y acumuladores

`for` por elementos, suma, contador, producto, mínimo, lista nueva.

Recorridos

Índices, izquierda ↔ derecha, dos extremos, dos listas, `%`, `while`.

Estructuras avanzadas

`for` dentro de `for`, recursividad, lista de diccionarios.

Cómo estudiar cada estructura

Cada sección de la guía trae las mismas cuatro partes.

Esqueleto

La forma general. Palabras como `condicion` son las piezas que tú cambias.

Ejemplo

Un caso mínimo que funciona, con su propia lista y su `return`.

Cómo reconocerla

Las pistas en el enunciado del problema.

⚠️ Error típico

Lo que más se equivoca en los exámenes.

Convención de nombres

Toda variable o parámetro que guarda una **lista** empieza con `lista_`: `lista_playlist`, `lista_resultados`. Con sólo leer el nombre sabes que puedes usar `for`, `len()` y `[i]`.

Todos los ejemplos están en `ejemplo_estructuras.py`: ejecútalo y compara cada `print` con su comentario.

1 · Función

Recibe datos (parámetros), hace algo con ellos y regresa un resultado.

Esqueleto

```
def nombre_funcion(par_1, par_2, par_n):
    # La lógica de la función va aquí
    return 0


print(nombre_funcion(1, 2, 3))  # 0
```

Partes de una función

- `def`, el nombre, los paréntesis y los dos puntos `:`
- Todo lo de adentro lleva 4 espacios de sangría
- Se ejecuta sólo cuando la **llamas**: `nombre_funcion(1, 2, 3)`

⚠️ Error típico

Usar `print` en lugar de `return`: la función "se ve" bien en la terminal, pero regresa `None`.

`return` vs `print`

`return` **regresa** el valor a quien llamó y **termina** la función. `print` sólo lo muestra en pantalla.

`return` vs `print`

En un partido, cada victoria vale 3 puntos. ¿Qué guarda la variable `puntos`?

Incorrecto

```
def puntos_por_victoria():
    print(3)


puntos = puntos_por_victoria()  # muestra 3
print(puntos)  # None
```

Correcto

```
def puntos_por_victoria():
    return 3


puntos = puntos_por_victoria()
print(puntos)  # 3
```

Con `print`

El 3 aparece en pantalla, pero la función regresa `None`: el valor se pierde.

Con `return`

El 3 regresa a quien llamó y queda guardado en `puntos`.

2 · Variables y tipos

El perfil de un jugador: cada dato en una variable de su tipo; al final, todos en un diccionario.

Python

```python
def crear_perfil():
    usuario = "ana_gamer"  # Texto (str)
    nivel = 42  # Entero (int)
    horas_jugadas = 310.5  # Decimal (float)
    es_premium = True  # Booleano (bool)
    lista_juegos = ["Minecraft", "Fortnite", "Mario Kart"]  # Lista (list)
    perfil = {  # Diccionario (dict): junta todos los datos del perfil
        "usuario": usuario,
        "nivel": nivel,
        "horas_jugadas": horas_jugadas,
        "es_premium": es_premium,
        "juegos": lista_juegos,
    }
    return perfil


perfil_ana = crear_perfil()
print(perfil_ana["usuario"])  # ana_gamer
print(perfil_ana["juegos"])  # ['Minecraft', 'Fortnite', 'Mario Kart']
```

= asigna, == compara

`nivel = 42` guarda; `nivel == 42` pregunta.

Lista: por posición

`lista_juegos[0]` es `"Minecraft"`

Diccionario: por llave

`perfil_ana["usuario"]` es `"ana_gamer"`

3 · `if` sin `else` · Uso 1: valor por defecto

Hay un bloque para cuando la condición se cumple y ninguno para cuando no.

Esqueleto

```
resultado = valor_por_defecto
if condicion:
    resultado = otro_valor
return resultado
```

La idea

La variable empieza con un **valor por defecto** y el `if` sólo la cambia cuando hace falta. Si no se cumple, el programa sigue con la siguiente línea.

Ejemplo: ¿qué canción suena primero en la playlist?

```
def primera_cancion(lista_playlist):
    cancion = None  # valor por defecto
    if len(lista_playlist) > 0:
        cancion = lista_playlist[0]  # sólo cambia si se cumple la condición
    return cancion


print(primera_cancion(["Tusa", "La Bachata", "Ojitos Lindos"]))  # Tusa
```

3 · `if` sin `else` · Uso 2: validar un edge case

El `if` revisa un edge case y, si se cumple, termina la función con `return`.

Esqueleto

```
if edge_case:
    return valor_para_el_edge_case  # la función termina aquí
# caso normal: sólo se llega aquí si el edge case no se cumplió
return resultado_normal
```

Ejemplo: la misma función, escrita como validación

```
def primera_cancion(lista_playlist):
    if len(lista_playlist) == 0:  # edge case: la playlist está vacía
        return None  # la función termina aquí
    return lista_playlist[0]  # caso normal: sólo se llega si hay canciones


print(primera_cancion([]))  # None
print(primera_cancion(["Tusa", "La Bachata", "Ojitos Lindos"]))  # Tusa
```

Que la condición se cumpla no es "algo extra": es la **salida de emergencia**. No hace falta `else`: el `return` ya terminó la función.

3 · `if` sin `else`: puntos clave

Validaciones al inicio

Van antes de usar los datos. Aquí protege a `lista_playlist[0]`, que con una playlist vacía lanzaría `IndexError`.

Una por cada edge case

Una función puede tener varias validaciones seguidas: playlist vacía, divisor en cero, dato inválido…

Cómo reconocerla

- Uso 1: "si pasa X, cambia / agrega / cuenta…" y no dice qué hacer en el otro caso
- Uso 2: "si la lista está vacía regresa…", "si el dato no es válido regresa…"

⚠️ Error típico

Olvidar el `return` dentro del `if`: la validación "detecta" el edge case, pero la función sigue de largo y truena en la línea de abajo.

4 · if / else

Dos caminos: si la condición se cumple, un bloque; si no, el otro. Siempre se ejecuta uno.

Versión 1

```
def es_mayor_edad(edad):
    if edad >= 18:
        return True
    else:
        return False


print(es_mayor_edad(20))  # True
```

Versión 2: la mayoría de edad como parámetro

```
def es_mayor_edad(edad, mayoria_edad):
    if edad >= mayoria_edad:
        return True
    else:
        return False


print(es_mayor_edad(20, 21))  # False
```

¿Por qué un parámetro?

La mayoría de edad no es igual en todos lados (en EE. UU., para algunas cosas, es 21).

Cómo reconocerla

"Si … regresa X, si no regresa Y".

⚠️ Error típico

Usar **>** cuando dice "mayor **o igual**": `es_mayor_edad(18)` daría `False`.

5 · if / elif / else

El plan del viernes depende de cuántos amigos se apuntan.

Python

```python
def plan_de_viernes(lista_amigos):
    if len(lista_amigos) == 0:
        return "maratón de series en casa"
    elif len(lista_amigos) == 1:
        return "ir al cine"
    elif len(lista_amigos) <= 5:
        return "noche de videojuegos"
    else:
        return "fiesta"


print(plan_de_viernes(["Ana", "Beto", "Caro"]))  # noche de videojuegos
```

En orden

Python revisa de arriba a abajo y ejecuta **sólo el primer** bloque que se cumple.

Cómo reconocerla

Rangos o categorías: bronce, plata, oro…

⚠️ Error típico: el orden

Lo más específico va primero. Con `<= 5` arriba, 0 amigos daría videojuegos.

6 · `if` anidados

La entrada a un concierto: sólo si tienes boleto se revisa tu edad para darte la pulsera.

Python

```python
def acceso_concierto(lista_con_boleto, nombre, edad):
    if nombre in lista_con_boleto:
        if edad >= 18:
            return "pasa con pulsera de mayor de edad"
        else:
            return "pasa sin pulsera"
    else:
        return "no tienes boleto"


print(acceso_concierto(["Ana", "Beto", "Caro"], "Beto", 17))  # pasa sin pulsera
```

Cada `else` con su `if`

El último `else` responde a "¿tienes boleto?": está a su misma altura.

`in`

`nombre in lista_con_boleto` es `True` si el nombre está en la lista.

Cómo reconocerla

"Si pasa X, revisa si también pasa Y" (años bisiestos: 4 → 100 → 400).

7 · and, or, not

Combinan varias condiciones en una sola.

Operador	Es True cuando…	Ejemplo
`a and b`	**las dos** son `True`	`edad >= 18 and nacionalidad == "mexicana"`
`a or b`	**al menos una** es `True`	`dia == "miercoles" or es_estudiante`
`not a`	`a` es `False` (lo voltea)	`not (cancion in lista_playlist)`

Si los mezclas, usa paréntesis

`and` se evalúa antes que `or`: `es_estudiante and (dia == "martes" or dia == "miercoles")`

7 · Ejemplos con and, or, not

and · para votar en México: 18 años y nacionalidad mexicana

```
def puede_votar(edad, nacionalidad):
    return edad >= 18 and nacionalidad == "mexicana"


print(puede_votar(18, "mexicana"))  # True
```

or · descuento en el cine: miércoles o credencial de estudiante

```
def descuento_en_cine(dia, es_estudiante):
    return dia == "miercoles" or es_estudiante


print(descuento_en_cine("sabado", True))  # True
```

not · ¿la canción no está en tu playlist?

```
def falta_en_playlist(lista_playlist, cancion):
    return not (cancion in lista_playlist)


print(falta_en_playlist(["Tusa", "La Bachata"], "Ojitos Lindos"))  # True
```

7 · `and`, `or`, `not`: puntos clave

Regresa la comparación

Una comparación ya es `True` o `False`: `return edad >= 18` en vez de `if … return True else … return False`.

`and` se detiene en el primer `False`

En `len(lista_playlist) > 0 and lista_playlist[0] == "Tusa"`, con una playlist vacía nunca se llega a `[0]`.

`if` anidados = `and`

Dos `if` anidados en los que **las dos** condiciones deben cumplirse son lo mismo que un `and`.

⚠️ Error típico

`dia == "sabado" or "domingo"` ✗

`dia == "sabado" or dia == "domingo"` ✓

Cada lado del `or` es una comparación completa.

Práctica: predice antes de ejecutar

Escribe la respuesta en tu cuaderno; después la comprobamos en la terminal.

1

```
acceso_concierto(["Ana"], "Beto", 20)
```

2

```
plan_de_viernes([])
```

3

```
es_mayor_edad(18, 21)
```

4

```
descuento_en_cine("miercoles", False)
```

5

```
puede_votar(25, "argentina")
```

6

```
primera_cancion([])
```

8 · `for` por elementos + acumulador

El `for` visita cada elemento; el acumulador guarda el resultado parcial en cada vuelta.

1 · Valor inicial

Antes del ciclo

2 · Recorrer

```
for elemento in
lista_elementos:
```

3 · Actualizar

Dentro del ciclo

4 · Regresar

Después del ciclo

Ejemplo: ¿cuántos minutos dura tu playlist?

```python
def duracion_playlist(lista_minutos):
    total = 0  # 1. valor inicial del acumulador
    for minutos in lista_minutos:  # 2. recorrer
        total = total + minutos  # 3. actualizar el acumulador
    return total  # 4. regresar, FUERA del ciclo


print(duracion_playlist([3, 4, 2, 5]))  # 14
```

Prueba de escritorio

vuelta	minutos	total
antes	—	0
1	3	3
2	4	7
3	2	9
4	5	**14**

8 · Errores típicos del acumulador

Con `[3, 4, 2, 5]` la respuesta correcta es 14. ¿Qué regresan estas versiones?

Incorrecto: el acumulador dentro del ciclo

```
def duracion_playlist(lista_minutos):
    for minutos in lista_minutos:
        total = 0  # se reinicia en cada vuelta
        total = total + minutos
    return total
```

Regresa 5

`total` vuelve a 0 en cada vuelta: sólo queda la última canción.

Incorrecto: el return dentro del ciclo

```
def duracion_playlist(lista_minutos):
    total = 0
    for minutos in lista_minutos:
        total = total + minutos
        return total  # sangría de más
```

Regresa 3

La función termina en la primera vuelta: sólo suma la primera canción.

9 · Tipos de acumuladores

Todos tienen los mismos 4 pasos. Cambia el valor inicial y cómo se actualiza.

Acumulador	Valor inicial	Actualización	Ejemplo
Suma	`0`	`total = total + x`	`duracion_playlist`
Contador	`0`	`contador = contador + 1` (dentro de un `if`)	`contar_victorias`
Producto	`1`	`producto = producto * x`	`combinaciones_outfit`
Mínimo / máximo	el primer elemento	`if x < mejor: mejor = x`	`mejor_vuelta`
Lista nueva	`[]`	`lista_nueva.append(x)`	`precios_al_alcance`
Varios a la vez	uno por cada dato	cada uno con su regla	`promedio_goles`

9 · Contador y producto

Contador: ¿cuántos partidos ganaste? (G ganado, P perdido, E empate)

```
def contar_victorias(lista_resultados):
    victorias = 0
    for resultado in lista_resultados:
        if resultado == "G":
            victorias = victorias + 1
    return victorias


print(contar_victorias(["G", "P", "G", "E", "G"]))  # 3
```

Contador

Empieza en `0` y suma `1` **dentro de un** `if`. Sin el `if`, cuenta todos los partidos.

Producto: 3 playeras, 2 pantalones y 4 tenis, ¿cuántos outfits?

```
def combinaciones_outfit(lista_opciones):
    combinaciones = 1
    for opciones in lista_opciones:
        combinaciones = combinaciones * opciones
    return combinaciones


print(combinaciones_outfit([3, 2, 4]))  # 24
```

Producto

Empieza en `1`. Si empezara en `0`, todo daría 0 (cero outfits).

9 · Mínimo y lista nueva

Mínimo: tu mejor vuelta en Mario Kart es la que tardó menos

```python
def mejor_vuelta(lista_segundos):
    mejor = lista_segundos[0]
    for segundos in lista_segundos:
        if segundos < mejor:
            mejor = segundos
    return mejor


print(mejor_vuelta([92, 87, 95]))  # 87
```

Mínimo

Empieza con el **primer elemento**. Con `0` regresaría 0, que ni está en la lista.

Lista nueva: ¿qué tenis te alcanzan con tu presupuesto?

```python
def precios_al_alcance(lista_precios, presupuesto):
    lista_alcanzables = []
    for precio in lista_precios:
        if precio <= presupuesto:
            lista_alcanzables.append(precio)
    return lista_alcanzables


print(precios_al_alcance([1200, 499, 899, 1500], 900))  # [499, 899]
```

Lista nueva

Empieza en `[]` y agrega con `append` sólo los que cumplen.

9 · Varios acumuladores a la vez

El promedio de goles por partido necesita la suma y cuántos partidos hubo.

Python

```python
def promedio_goles(lista_goles):
    total = 0
    partidos = 0
    for goles in lista_goles:
        total = total + goles
        partidos = partidos + 1
    return total / partidos


print(promedio_goles([2, 0, 3, 1]))  # 1.5
```

Dos acumuladores

`total` suma los goles; `partidos` cuenta cuántos hubo.

Resultado

`6 / 4` = `1.5` goles por partido.

Cómo reconocer un acumulador

"suma", "cuenta", "total", "cuántos", "promedio", "el mejor", "regresa una lista con los que…"

10 · `for` con índices

Cuando importa la posición del elemento: dónde está, su vecino, dos listas a la vez.

Ejemplo: ¿en qué posición de la playlist está una canción?

```
def posicion_de_cancion(lista_playlist, cancion):
    for i in range(0, len(lista_playlist), 1):  # i = 0, 1, 2, ..., len - 1
        if lista_playlist[i] == cancion:
            return i  # termina la función en cuanto la encuentra
    return -1  # sólo llega aquí si revisó todo y no la encontró


print(posicion_de_cancion(["Tusa", "La Bachata", "Ojitos Lindos"], "La Bachata"))  # 1
```

Valores vs posiciones

`for cancion in lista_playlist` da valores; `for i in range(…)` da posiciones.

`return` temprano

Termina en cuanto la encuentra. El `return -1` va **después** del ciclo.

⚠️ Error típico

`else: return -1` dentro del ciclo: se rinde tras la primera canción.

Cómo leer `range(inicio, fin, incremento)`

`range(0, len(lista_playlist), 1)` con `lista_playlist = ["Tusa", "La Bachata", "Ojitos Lindos"]`

inicio · `0`

La primera posición.

fin · `len(lista_playlist)`

Vale 3 y **no se incluye**: el ciclo para antes.

incremento · `1`

Avanza de 1 en 1.

posición **0**	posición **1**	posición **2**	posición **3**
`"Tusa"`	`"La Bachata"`	`"Ojitos Lindos"`	no existe

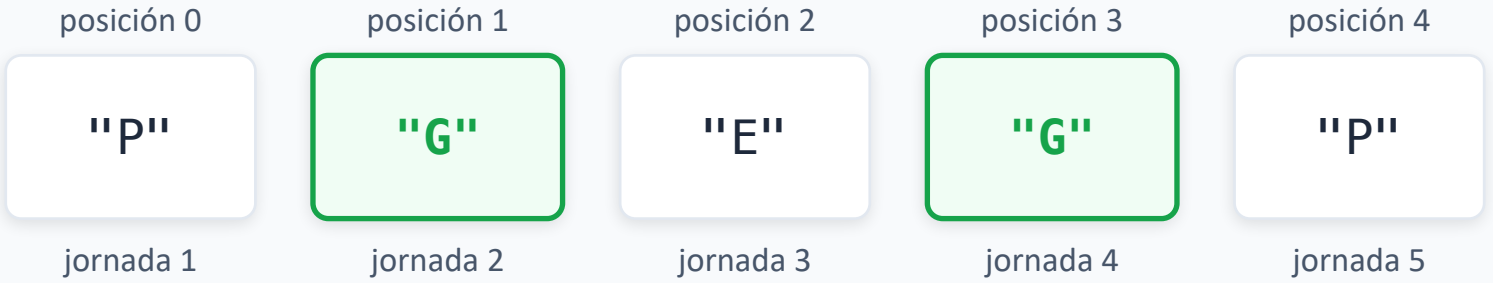
`i` = 0, 1, 2 ✓

La última posición es `len(lista_playlist) - 1`

`lista_playlist[3]` → `IndexError`

11 · Izquierda → derecha y derecha → izquierda

En los resultados de la temporada, ¿en qué jornada fue tu primera victoria y en cuál la última?



→ primera `"G"`: posición 1 = **jornada 2**

última `"G"`: posición 3 = **jornada 4** ←

Izquierda → derecha · "el primero que…"

```
range(0, len(lista_resultados), 1)
```

visita 0, 1, 2, 3, 4

Derecha → izquierda · "el último que…"

```
range(len(lista_resultados) - 1, -1, -1)
```

visita 4, 3, 2, 1, 0

Hacia atrás: empieza en la **última** posición, para **antes de** −1 (o sea, incluye el 0) y avanza de **-1 en -1**.

⚠️ `range(len(lista_resultados), 0, -1)` empieza en una posición que no existe y nunca visita el 0.

11 · La misma búsqueda en las dos direcciones

Como la jornada 1 está en la posición 0, la función regresa `i + 1`.

Izquierda → derecha: la primera victoria

```
def jornada_primera_victoria(lista_resultados):
    for i in range(0, len(lista_resultados), 1):  # 0, 1, 2, ..., len - 1
        if lista_resultados[i] == "G":
            return i + 1  # las posiciones empiezan en 0, las jornadas en 1
    return -1  # no hubo ninguna victoria


print(jornada_primera_victoria(["P", "G", "E", "G", "P"]))  # 2
```

Derecha → izquierda: la última victoria

```
def jornada_ultima_victoria(lista_resultados):
    for i in range(len(lista_resultados) - 1, -1, -1):  # len - 1, ..., 2, 1, 0
        if lista_resultados[i] == "G":
            return i + 1  # las posiciones empiezan en 0, las jornadas en 1
    return -1  # no hubo ninguna victoria


print(jornada_ultima_victoria(["P", "G", "E", "G", "P"]))  # 4
```

12 · Al mismo tiempo: dos extremos

¿Tu gamertag se lee igual al derecho y al revés? Un texto se recorre igual que una lista.

Python

```python
def es_palindromo(gamertag):
    izquierda = 0
    derecha = len(gamertag) - 1
    while izquierda < derecha:
        if gamertag[izquierda] != gamertag[derecha]:
            return False
        izquierda = izquierda + 1
        derecha = derecha - 1
    return True


print(es_palindromo("otto"))  # True
```



vuelta	izq.	der.	compara
1	0	3	o con o ✓
2	1	2	t con t ✓
—	2	1	se detiene

Se mueven **en la misma vuelta** hasta encontrarse. No sabemos cuántas vueltas habrá → `while`.

⚠️ Si olvidas mover uno de los índices, el ciclo nunca termina.

12 · Al mismo tiempo: dos listas con el mismo índice

La quiniela: comparas tu pronóstico de cada partido con el resultado real (L local, E empate, V visitante).

Python

```python
def aciertos_quiniela(lista_pronosticos, lista_resultados):
    aciertos = 0
    for i in range(0, len(lista_resultados), 1):
        if lista_pronosticos[i] == lista_resultados[i]:
            aciertos = aciertos + 1
    return aciertos


print(aciertos_quiniela(["L", "E", "V", "L"], ["L", "V", "V", "L"]))  # 3
```

	i = 0	i = 1	i = 2	i = 3	
lista_pronosticos	"L"	"E"	"V"	"L"	
lista_resultados	"L"	"V"	"V"	"L"	
	✓	✗	✓	✓	**3** aciertos

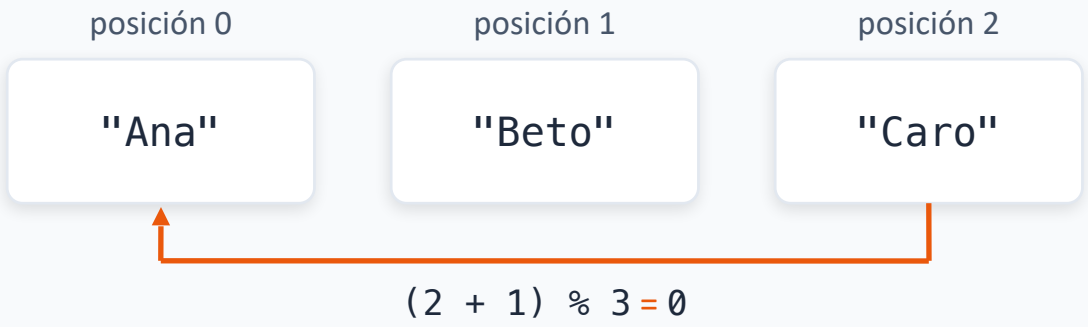
13 · Recorrido circular con %

En el coche cada amigo elige una canción por turno. Empieza Caro (posición 2): ¿quién elige las siguientes 4?

Python

```python
def turnos_dj(lista_amigos, posicion_inicial, cantidad_canciones):
    lista_turnos = []
    posicion = posicion_inicial
    for turno in range(0, cantidad_canciones, 1):
        lista_turnos.append(lista_amigos[posicion])
        posicion = (posicion + 1) % len(lista_amigos)
    return lista_turnos


print(turnos_dj(["Ana", "Beto", "Caro"], 2, 4))  # ['Caro', 'Ana', 'Beto', 'Caro']
```



Cómo reconocerla

`x % n` siempre da entre `0` y `n − 1`. "Por turnos", "da la vuelta", Pacman, horas, días de la semana. Hacia atrás: `(0 − 1) % 3 = 2`.

14 · `while`: las 3 partes

Repite mientras la condición sea `True`. A diferencia del `for`, tú controlas el inicio, la condición y el avance.

1 · Inicio

```
i = 0
```

2 · Condición para seguir

```
while i <
len(lista_minutos):
```

3 · Avance

```
i = i + 1
```

Ejemplo 1: el mismo recorrido que un for (la duración de la playlist)

```python
def duracion_con_while(lista_minutos):
    total = 0
    i = 0  # 1. inicio
    while i < len(lista_minutos):  # 2. condición para seguir
        total = total + lista_minutos[i]
        i = i + 1  # 3. avance (sin esta línea, el ciclo es infinito)
    return total


print(duracion_con_while([3, 4, 2, 5]))  # 14
```

⚠️ Sin el avance, el ciclo es **infinito**. La plataforma lo corta a los 10 s.

14 · `while`: detenerse cuando se cumple algo

Ahorras cada semana para un boleto de $500. El ciclo para en cuanto juntas el dinero.

Python

```python
def semanas_para_boleto(lista_ahorros, precio):
    total = 0
    semanas = 0
    while semanas < len(lista_ahorros) and total < precio:
        total = total + lista_ahorros[semanas]
        semanas = semanas + 1
    if total < precio:  # edge case: se acabaron las semanas y no alcanzó
        return -1
    return semanas


print(semanas_para_boleto([200, 150, 300, 250], 500))  # 3
print(semanas_para_boleto([100, 50], 500))  # -1
```

vuelta	total	¿sigue?
1	200	sí
2	350	sí
3	650	no

Regresa 3

En la vuelta 3 el total pasa de $500. Con `[100, 50]` nunca alcanza: regresa `-1`.

El `-1` es un **edge case**, validado con un `if` sin `else` (sección 3). La revisión `semanas < len(lista_ahorros)` va **primero** en el `and`: protege a `lista_ahorros[semanas]`.

¿`for` o `while`?

Usa `for` cuando…

- Vas a recorrer **toda** la lista
- Sabes cuántas vueltas habrá

Usa `while` cuando…

- Te detienes cuando **se cumple algo**
- No sabes cuántas vueltas habrá
- Mueves dos índices de forma distinta (dos extremos, búsqueda binaria)

Cómo reconocer un `while`

"hasta que…", "mientras…", "¿cuántas semanas necesitas?", "desde los dos extremos"

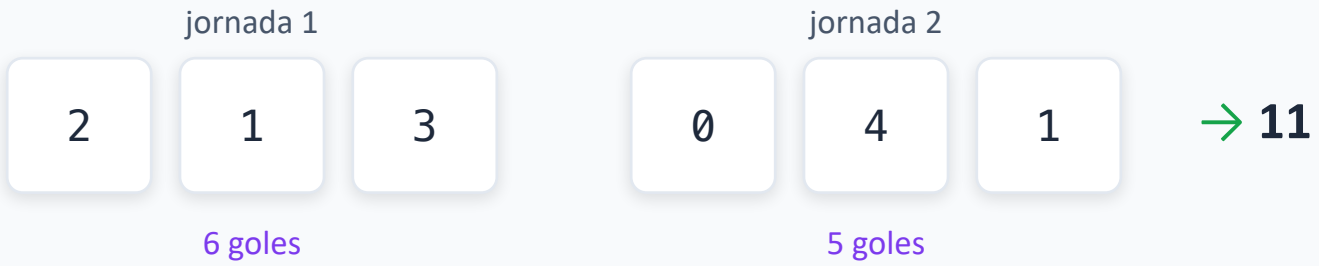
15 · `for` dentro de `for`: lista de listas

Un torneo tiene jornadas, y cada jornada tiene partidos. ¿Cuántos goles hubo en todo el torneo?

Python

```python
def goles_del_torneo(lista_jornadas):
    total = 0
    for lista_partidos in lista_jornadas:  # ciclo de afuera: cada jornada
        for goles in lista_partidos:  # ciclo de adentro: cada partido de esa jornada
            total = total + goles
    return total


print(goles_del_torneo([[2, 1, 3], [0, 4, 1]]))  # 11
```



La regla

Por **cada vuelta** del ciclo de afuera, el de adentro da **todas** sus vueltas.

15 · `for` dentro de `for`: cada pareja

Quieres comprar dos videojuegos y tienes $800. ¿Cuántas parejas distintas te alcanzan?

Python

```python
def parejas_de_juegos(lista_precios, presupuesto):
    parejas = 0
    for i in range(0, len(lista_precios), 1):
        for j in range(i + 1, len(lista_precios), 1):  # j empieza después de i
            if lista_precios[i] + lista_precios[j] <= presupuesto:
                parejas = parejas + 1
    return parejas


print(parejas_de_juegos([300, 500, 200, 700], 800))  # 3
```

Las parejas

300 + 500, 300 + 200 y 500 + 200 → **3**

¿Por qué `i + 1`?

No emparejar un juego consigo mismo ni contar dos veces la misma pareja.

Big O

n × n vueltas: **O(n²)**

⚠️ Error típico

`i` en los dos ciclos: el de adentro pisa al de afuera.

16 · Recursividad

Una función que se llama a sí misma con un problema más pequeño, hasta llegar a uno que se responde directo.

Esqueleto: las 2 partes obligatorias

```
def funcion_recursiva(problema):
    if es_el_caso_base:  # 1. caso base: se responde sin llamarse otra vez
        return respuesta_directa
    else:  # 2. caso recursivo: se llama con un problema MÁS PEQUEÑO
        return combinar(parte, funcion_recursiva(problema_mas_pequeno))
```

1 · Caso base

Se responde **sin** llamarse otra vez. Es el problema más pequeño.

2 · Caso recursivo

Se llama con un problema **más pequeño** y combina el resultado.

Cómo reconocerla: el enunciado dice "recursivo", o el problema se define en términos de sí mismo: "el factorial de n es n × el factorial de n - 1".

16 · Ejemplo: recortando la lista

La duración de la playlist es "la primera canción + la duración del resto". `lista_minutos[1:]` es la lista sin su primer elemento.

Python

```python
def duracion_recursiva(lista_minutos):
    if len(lista_minutos) == 0:  # caso base: el problema más pequeño
        return 0
    else:  # caso recursivo: la primera canción + la duración del resto
        return lista_minutos[0] + duracion_recursiva(lista_minutos[1:])


print(duracion_recursiva([3, 4, 2, 5]))  # 14
```

Las llamadas bajan hasta el caso base y los resultados suben

```
duracion_recursiva([3, 4, 2, 5])
= 3 + duracion_recursiva([4, 2, 5])
= 3 + 4 + duracion_recursiva([2, 5])
= 3 + 4 + 2 + duracion_recursiva([5])
= 3 + 4 + 2 + 5 + duracion_recursiva([])
= 3 + 4 + 2 + 5 + 0                    ← caso base: playlist vacía
= 14
```

16 · Con un índice, y errores típicos

Otra forma: avanza la posición en lugar de recortar la lista.

Python

```python
def duracion_recursiva_indice(lista_minutos, i):
    if i == len(lista_minutos):  # caso base: ya no quedan canciones
        return 0
    else:
        return lista_minutos[i] + duracion_recursiva_indice(lista_minutos, i + 1)


print(duracion_recursiva_indice([3, 4, 2, 5], 0))  # 14
```

⚠️ Sin caso base

O con uno que nunca se alcanza: `RecursionError`.

⚠️ El mismo problema

Llamarse con la **misma** lista en lugar de una más pequeña: nunca termina.

⚠️ Sin `return`

Olvidarlo antes de la llamada recursiva: el resultado se pierde y regresa `None`.

17 · Lista de diccionarios

¿Quiénes del equipo metieron 5 goles o más en la temporada?

Python

```python
def goleadores(lista_jugadores):
    lista_goleadores = []
    for jugador in lista_jugadores:
        if jugador["goles"] >= 5:
            lista_goleadores.append(jugador["nombre"])
    return lista_goleadores


lista_equipo = [
    {"nombre": "Ana", "goles": 7},
    {"nombre": "Beto", "goles": 2},
    {"nombre": "Caro", "goles": 5},
]
print(goleadores(lista_equipo))  # ['Ana', 'Caro']
```

jugador["nombre"]	jugador["goles"]	¿≥ 5?
"Ana"	7	✓
"Beto"	2	✗
"Caro"	5	✓

La única novedad

Cada `jugador` es un diccionario: sus datos se piden por **llave**.

⚠️ **Error típico**

`jugador[goles]`: la llave va entre comillas y escrita igual.

18 · ¿Qué estructura uso? (1/2)

Si el problema dice…	Estructura	Sección
"si pasa X, cambia / agrega…" (sin otro caso)	`if` sin `else`	3
"si la lista está vacía / el dato no es válido, regresa…"	`if` de validación con `return`	3
"si … si no …"	`if` / `else`	4
rangos, categorías, 3 o más casos	`if` / `elif` / `else`	5
"si X, y además si Y…"	`if` anidados **o** `and`	6, 7
"todos deben cumplir", "a la vez"	`and`	7
"cualquiera de", "alguno de"	`or`	7
"no es", "ninguno"	`not`	7
suma, total, promedio	`for` + acumulador (empieza en 0)	8, 9
"cuántos cumplen…"	`for` + `if` + contador	9
el mayor / el menor / el mejor	acumulador con el primer elemento	9

18 · ¿Qué estructura uso? (2/2)

Si el problema dice…	Estructura	Sección
"regresa una lista con los que…"	acumulador `[]` + `append`	9
"en qué posición", "el índice"	`for i in range(0, len(lista), 1)`	10
"el primero que…"	izquierda → derecha + `return` temprano	10, 11
"el último que…", "al revés"	derecha → izquierda	11
palíndromo, "desde los dos extremos"	dos índices + `while`	12
comparar dos listas posición por posición	un índice para las dos listas	12
turnos, "da la vuelta", horas, días	`% len(lista_elementos)`	13
"hasta que…", "mientras…"	`while`	14
tabla, lista de listas, cada pareja, repetidos	`for` dentro de `for`	15
"de forma recursiva"	caso base + caso recursivo	16
registros con nombre, goles, precio…	lista de diccionarios	17

Antes de entregar cualquier ejercicio, revisa

1

¿Mi función tiene `return`, y está **fuera** del ciclo, a menos que quiera terminar antes?

2

¿Mi acumulador empieza **antes** del ciclo y con el valor correcto (0, 1, `[]`, el primer elemento)?

3

¿Probé con una lista **vacía**, con **un solo** elemento y con el buscado al **inicio** y al **final**?

4

¿Uso >= o > según lo que dice el enunciado?

5

Si uso `while`: ¿algo cambia en cada vuelta para que el ciclo termine?

Práctica: ¿qué estructura usarías?

Sólo identifica la estructura y escribe su esqueleto; no hace falta resolverlo completo.

A

¿Cuántas canciones de tu playlist duran más de 4 minutos?

B

¿En qué jornada fue tu última derrota?

C

Si la lista de precios está vacía regresa 0; si no, el precio más barato.

D

Cada amigo pone una canción por turno: ¿quién elige la canción 10?

E

¿Cuántas semanas necesitas para juntar $2,000 para un videojuego?

F

¿Hay dos gamertags repetidos en la lista de tu clan?

Cierre

Decisiones

`if`, `elif`, `else`, `and` / `or` / `not`: eligen qué camino seguir.

Ciclos

`for`, `while` y acumuladores: repiten y van guardando el resultado.

Recorridos

Índices, direcciones, `%`, recursividad: cómo visitar los datos.

Para el parcial

Estudia con `guia_estructuras.pdf`. Ejecuta `ejemplo_estructuras.py` y compara cada `print` con su comentario; después escribe cada esqueleto de memoria.